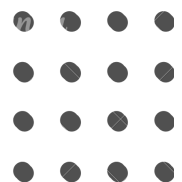




VMES Grinds

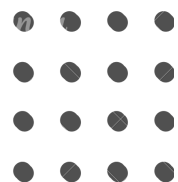




VMES Grinds

Pilot Program - Week 6 - Day 2

Graph, BFS, DFS





797. All Paths From Source to Target





797. All Paths From Source to Target

Solved ✓

Medium

Topics

Companies

Given a directed acyclic graph (**DAG**) of n nodes labeled from 0 to $n - 1$, find all possible paths from node 0 to node $n - 1$ and return them in **any order**.

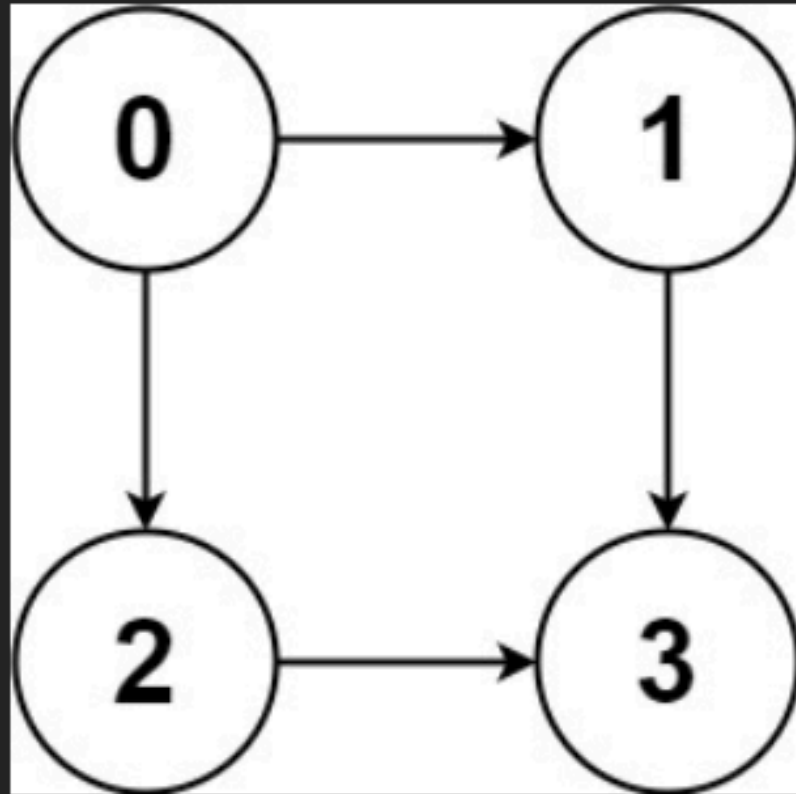
The graph is given as follows: `graph[i]` is a list of all nodes you can visit from node i (i.e., there is a directed edge from node i to node `graph[i][j]`).

Constraints:

- `n == graph.length`
- `2 <= n <= 15`
- `0 <= graph[i][j] < n`
- `graph[i][j] != i` (i.e., there will be no self-loops).
- All the elements of `graph[i]` are **unique**.
- The input graph is **guaranteed** to be a **DAG**.



Example 1:

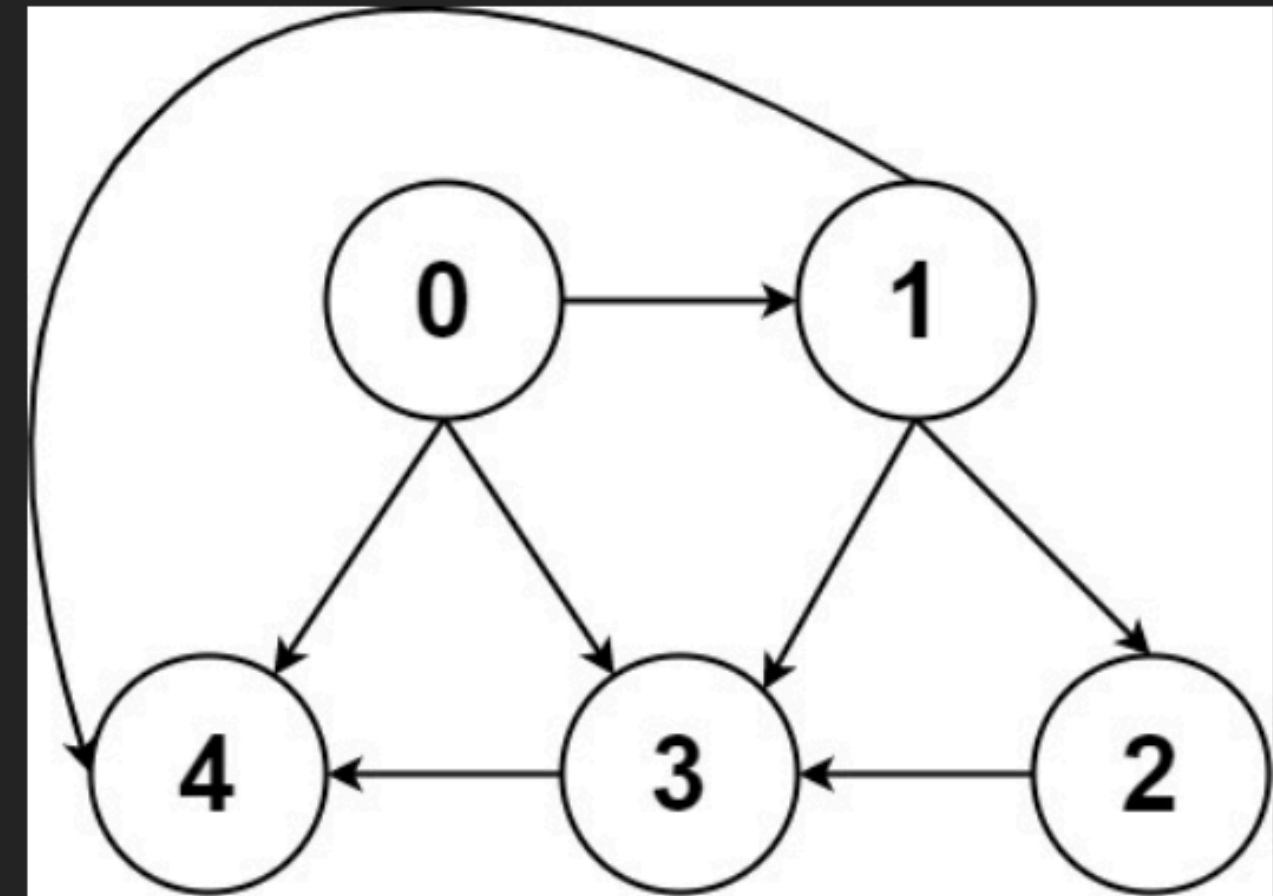


Input: graph = [[1,2],[3],[3],[]]

Output: [[0,1,3],[0,2,3]]

Explanation: There are two paths: 0 → 1 → 3 and 0 → 2 → 3.

Example 2:



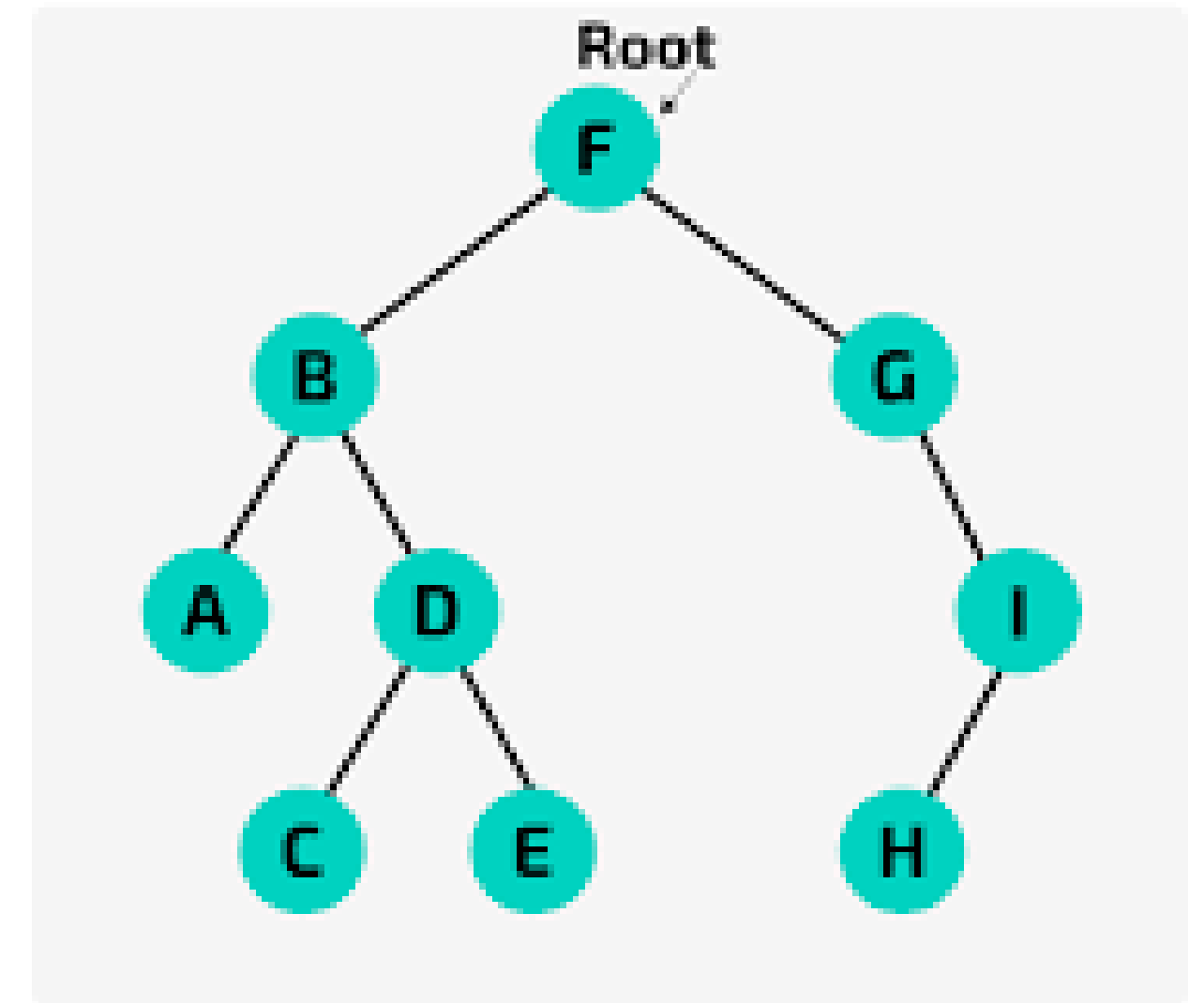
Input: graph = [[4,3,1],[3,2,4],[3],[4],[]]

Output: [[0,4],[0,3,4],[0,1,3,4],[0,1,2,3,4],[0,1,4]]



Explanation





Thought Process

- Branching out & check → BFS or DFS
- BFS is memory heavy + not looking for shortest path → Use DFS



Code





```
class Solution:
    def allPathsSourceTarget(self, graph):
        res = []
        n = len(graph)

        def dfs(node, path):
            if node == n - 1:
                res.append(path[:])
                return

            for nei in graph[node]:
                dfs(nei, path + [nei])

        dfs(0, [0])
        return res

sol = Solution()
print(sol.allPathsSourceTarget([[1, 2], [3], [3], []]))
print(sol.allPathsSourceTarget([[4, 3, 1], [3, 2, 4], [3], [4], []]))
```



Time and Space Complexity Analysis





```
class Solution:
    def allPathsSourceTarget(self, graph):
        res = []
        n = len(graph)

        def dfs(node, path):
            if node == n - 1:
                res.append(path[:])
                return

            for nei in graph[node]:
                dfs(nei, path + [nei])

        dfs(0, [0])
        return res

sol = Solution()
print(sol.allPathsSourceTarget([[1, 2], [3], [3], []]))
print(sol.allPathsSourceTarget([[4, 3, 1], [3, 2, 4], [3], [4], []]))
```

← Space - $O(P \times L^2)$

← Time - $O(P * L)$

P = number of paths
L = average path length



Optimized Code





```
class Solution:
    def allPathsSourceTarget(self, graph: list[list[int]]) -> list[list[int]]:
        res = []
        sol = []
        n = len(graph)

        def dfs(node):
            sol.append(node)
            if node == n - 1:
                res.append(sol.copy())
            else:
                for nei in graph[node]:
                    dfs(nei)
            sol.pop()

        dfs(0)
        return res

sol = Solution()
print(sol.allPathsSourceTarget([[1, 2], [3], [3], []]))
print(sol.allPathsSourceTarget([[4, 3, 1], [3, 2, 4], [3], [4], []]))
```




Time and Space Complexity Analysis





```
class Solution:
    def allPathsSourceTarget(self, graph: list[list[int]]) -> list[list[int]]:
        res = []
        sol = []
        n = len(graph)

        def dfs(node):
            sol.append(node)
            if node == n - 1:
                res.append(sol.copy())
            else:
                for nei in graph[node]:
                    dfs(nei)
            sol.pop()

        dfs(0)
        return res

sol = Solution()
print(sol.allPathsSourceTarget([[1, 2], [3], [3], []]))
print(sol.allPathsSourceTarget([[4, 3, 1], [3, 2, 4], [3], [4], []]))
```

← Space - $O(L)$

← Time - $O(P * L)$

P = number of paths
 L = average path length



Key Takeaway





Both solutions are $O(P \times L)$ time, but:

- **Backtracking (optimal) version**
 - uses (Stack) one mutable path
 - is more memory-efficient and faster
- **Path-copy version**
 - creates many temporary lists
 - is simpler but less optimal



Thank You !